DA3408: Assignment 5

SwiftParcel ETA service served four ways

Ananya Kishore Roll no. DA24B035

Machine and setup

All measurements were taken on a laptop with 12 CPU cores and about 8 GB RAM, Ubuntu, Python 3.14 (12th Gen Intel Core i7-1255U). The load generator and the servers ran on the same machine. Everything runs on CPU. The Ray application was started with `num_cpus=2` as required. Code is in `partA` to `partD` of the repository, and `README.md` lists the commands. Data and model bundles are regenerated with `python eta_data.py`.

Sources: Structural ideas were adapted from the Lecture 10 notebooks: REST_API_Serving (`rest_app.py`: model loaded once, pydantic schema, sync handler), MLflow_Serving (`mlflow_register.py`: pyfunc wrapper, signature, aliases, promotion gate) and Ray_Serve (`ray_serve_app.py`: deployment, router, `build` function). The starter files `eta_data.py` and `load_test.py` are used unchanged.

Part A: REST service

A1. `partA/eta_app.py` loads the bundle from `ETA_MODEL` (`data/eta_v1.joblib`) once at import into the module-level variable `model`. A pydantic model enforces every range, and a model validator rejects `distance_km > 150` with `n_stops > 20` with a message containing "infeasible". The batch size is limited to 1 to 500. One HTTP middleware echoes or generates `X-Request-ID` on every response, including 422 and 500, and records the Prometheus counter and histogram (requests to `/metrics` are skipped). A custom validation handler returns the required 422 body. Inference errors return a generic 500 body `{"error": "internal_error", "request_id": ...}` with no exception text.

A2. `partA/test_eta_app.py` has 16 test functions (21 cases with parametrisation) using `TestClient`: a valid request, batch order (compared with direct model output, and reversed), healthz, three kinds of invalid input (out of range, missing field, wrong type) plus six range edges, the infeasible rule and its boundary (150 km and 20 stops is accepted), empty batch, 500 accepted and 501 rejected, request-id echo, generation and presence on errors, the metrics format (counter deltas, buckets, `+Inf`, `_sum`, `_count`, `/metrics` not counted), and a generic 500 with a monkeypatched model. All 21 pass.

A3.1 Load test (v1, one uvicorn worker, batch 1, 4 s per level, `partA/loadtest_results.json`): The last column is the mean time in system predicted by Little's Law, $W = L/\lambda$.

Concurrency L	Requests	λ (rps)	p50 (ms)	p95 (ms)	p99 (ms)	Errors	L/λ (ms)
1	1776	444.0	2.2	2.6	2.9	0	2.25
2	2184	546.0	3.3	4.7	5.8	0	3.66
4	2006	501.5	7.6	10.6	14.1	0	7.98
8	1356	339.0	23.0	37.7	47.4	0	23.6
16	1499	374.8	40.4	64.9	106.4	0	42.7

A3.2 Saturation and capacity: In a closed loop the number of clients fixes L, so Little's Law gives the mean latency $W = L/\lambda$. The predicted W agrees with the measured latency at every level (for example 7.98 vs 7.6 ms and 42.7 vs 40.4 ms, the mean being slightly above p50 because of the right tail), so the measurements are consistent. Throughput rises from 444 to 546 rps between $L = 1$ and $L = 2$, then stops rising and even falls, while latency grows roughly in proportion to L. This is the signature of saturation, $W \approx L/\lambda_{\max}$: extra clients only wait in the queue. The **saturation point is $L \approx 2$ at about 550 rps**. Throughput drops instead of staying flat because one Python process (one GIL) does the validation, DataFrame construction and prediction, and the Python load generator competes for the same CPU. Every level met the SLO p95 $\leq$ 80 ms (worst 64.9 ms at $L = 16$), so the SLO is not the binding limit, the CPU of one worker is. The **capacity under the SLO is therefore about 550 rps** (546 rps measured at $L = 2$ with p95 4.7 ms). Higher concurrency only adds

latency (p99 above 100 ms at $L = 16$), and extrapolating p95 at about 4 ms per extra client, the SLO would break at roughly 20 concurrent clients.

A3.3 Cost per row (concurrency 1):

Batch	Requests in 4 s	p50 per request (ms)	ms per row	Rows per second
1	1795	2.2	2.2	about 449
100	1344	2.7	0.027	about 33 600

A row is about 81 times cheaper in a batch of 100. Almost all of the 2.2 ms is fixed per-request overhead (HTTP round trip, JSON parsing, middleware, pydantic validation, building a DataFrame, sklearn input checks and serialising the response). The Ridge prediction itself is a vectorised product that costs microseconds per row. A batch pays the fixed cost once, so 100 rows take only 0.5 ms more than one row. The price is a higher latency for that request, and in practice rows must wait to form a batch.

A3.4 Change: two uvicorn workers (`-workers 2`): One worker is limited to about one core by the GIL, and the machine has 12 cores, so two processes should roughly double peak capacity.

L	rps, 1 worker	rps, 2 workers	Speed-up	p95, 1 worker	p95, 2 workers	p99, 2 workers
1	444.0	440.0	0.99	2.6	2.7	3.3
2	546.0	605.2	1.11	4.7	4.2	4.9
4	501.5	879.0	1.75	10.6	5.8	6.5
8	339.0	824.2	2.43	37.7	11.8	14.0
16	374.8	780.8	2.08	64.9	25.8	29.9

Result: **peak capacity rose from 546 to 879 rps, a factor of 1.61, not a full doubling**, and the saturation point moved to $L \approx 4$ (two requests per worker). At equal high concurrency the gain is 2.1 to 2.4 times, but partly because the single worker degraded past saturation. Tail latency improved the most (p95 at $L = 16$ fell from 64.9 to 25.8 ms). The peak did not double mainly because the load generator is a single Python process on the same machine and limits how fast requests are sent, and because low concurrency cannot keep both workers busy. A side effect is that `/metrics` becomes per process.

A3.5 Why p95/p99: Latency is right-skewed with a long tail from queueing, garbage collection and scheduling, so the mean hides the slow requests that a real share of users experiences (at $L = 16$, p50 is 40 ms but p99 is 106 ms). SLOs are promises about almost every request, and the tail is what degrades first under load, so the high percentiles decide user experience and capacity.

Part B: MLflow registry, gate and shadow

B1. `EtaPyfunc` loads the bundle in `load_context` and returns a DataFrame with `eta_minutes` (one decimal) and `model_version`. Both models were logged with an inferred signature (seven float64 inputs, outputs `eta_minutes` double and `model_version` string), a two-row input example and pinned `pip_requirements` (`mlflow==3.15.1`, `scikit-learn==1.9.0`, `pandas==2.3.3`, `numpy==2.5.2`, `joblib==1.5.3`, `cloudpickle==3.1.2`). The tracking store is SQLite (`MLFLOW_TRACKING_URI=sqlite:///mlflow.db`).

Version	Alias	Bundle	Model	Training MAE (min)
1	prod	v1	Pipeline (StandardScaler, Ridge)	11.66
2	candidate	v2	GradientBoostingRegressor	4.99

B2. `decide` is pure and checks the four criteria. I read `max_slice_regression` as relative, like `min_rel_gain`: the candidate's slice MAE may be at most 5% above prod's. `promotion_gate` evaluates both aliases on `make_data(2000, seed=99)` and measures p95 over 200 single-row `predict` calls after a warm-up. `test_gate.py` has 16 tests, including values just inside, exactly at and just outside every threshold, a model that is better overall but worse on a slice, custom thresholds and purity. All 16 pass. Output (`partB/gate_output.txt`):

Fresh-data metric	prod (v1)	candidate (v2)
MAE (min)	11.955	6.105
MAE, rush hour (hours 7 to 9 and 16 to 19)	19.680	8.580
MAE, rain = 1	11.623	6.285
Single-row p95 (ms)	0.905	0.939

Criterion	Default	`min_rel_gain=0.60`
Relative MAE gain 0.489	pass (≥ 0.03)	**fail** (≥ 0.60)
Candidate p95 0.94 ms	pass (≤ 10)	pass
Rush 8.58	pass (≤ 20.66)	pass
Rain 6.29	pass (≤ 12.20)	pass
Promote	**true**	**false**

With 0.60 only the gain criterion fails, since a 49% improvement is large but below the 60% demanded. The p95 in the second run was 0.893 ms, a normal timing difference. The MAEs are identical because the data is deterministic.

B3. Shadow comparison (`partB/shadow_output.txt`). Prod was served on port 5101 and candidate on port 5102 with `mlflow models serve`. The same 200 rows of `data/eta_fresh.csv` were sent one row per request to both servers, alternating which server was called first.

(i) Mean absolute difference between predictions	9.48 min
(ii) Requests whose predictions differ by more than 10 min	35.0%
(iii) p95 latency, prod / candidate	4.51 ms / 4.18 ms
Additional: MAE against the true ETA on these 200 rows, prod / candidate	11.42 / 6.11 min

I would promote v2, but through a canary rather than an instant switch. It disagrees with v1 by 9.5 minutes on average and by more than 10 minutes on a third of requests, and the ground truth shows these are v1's errors (MAE 11.4 vs 6.1, with the largest gain on rush-hour deliveries), at the same serving latency. Because so many customers will see their ETA change, a gradual rollout with abort criteria is the safer path. Moving an alias does not change what a running server returns because `mlflow models serve` resolves `models:/eta-model@prod` to a concrete version once at start-up and keeps that model in memory, so the alias is registry metadata and not a live pointer. A running service can pick up the new version without dropping requests by (1) a blue-green or rolling replacement, where a new server that resolves the alias to v2 is started, traffic moves to it once its readiness probe passes and the old server is drained, or (2) an in-process hot reload, where a background thread watches the alias and atomically swaps the model reference while in-flight requests finish on the old model.

Part C: Ray Serve sticky canary

C1. `EtaModel` loads its bundle once per replica. Prod runs 2 replicas, and the candidate uses `autoscaling_config` with min 1 and max 3. The FastAPI router takes `customer_id` and `instances` and returns the model output plus `arm`. The arm is v2 when the SHA-256 bucket of the customer id (first 8 bytes divided by 2^{64}) is below `weight`, or when the header `X-Beta: 1` is present. Raising the weight only widens the v2 interval, so no customer moves back to prod. Replicas reserve 0.25 CPU and the router 0.1 CPU, so the worst case is $0.1 + 2 \times 0.25 + 3 \times 0.25 = 1.35$ CPUs, which fits on 2 CPUs. `build(weight=0.1, extra_work_ms=0.0)` returns the application.

C2. `verify_ray.py` runs on Ray with 2 CPUs, weight 0.2 and 1000 distinct customer ids. All three deployments were healthy and every assertion passed (`partC/verify_output.txt`).

Check	Evidence	Result
(a) Share on candidate	189 of 1000, share 0.189 (allowed 0.16 to 0.24)	pass
(b) Second call, 200 customers	200 of 200 on the same arm (39 on v2)	pass
(c) After redeploy (delete and run)	1000 of 1000 on the same arm	pass
(d) Weight 0.2 to 0.5	share 0.490, 301 moved to v2, 0 moved back to v1	pass
(e) `X-Beta: 1`, 50 prod customers	50 of 50 served by v2, 50 of 50 back on v1 without it	pass

C3.

1. Python salts `hash()` for strings with a random seed per process (hash randomisation). Every Ray replica is a separate process and every restart or redeploy creates new ones, so the same customer would land in different buckets on different replicas or after a redeploy, which breaks stickiness. A cryptographic hash such as SHA-256 of the id gives the same value in every process and on every machine.
2. For the product, a customer who refreshes the tracking page must see one consistent ETA, not values that jump between two models, which looks broken and erodes trust. For the analysis, the customer is the experimental unit, so each customer's outcomes (accuracy, complaints, late deliveries) belong to exactly one arm and the comparison is not contaminated by customers who saw both models.

3. `target_ongoing_requests=2` keeps the queue per replica short, because a prediction takes a few milliseconds and two waiting requests add little latency against an 80 ms SLO. `upscale_delay_s=5` reacts to a real burst within seconds but ignores single spikes, while `downscale_delay_s=60` with a 15 s look-back (Ray requires it to exceed the 10 s metrics interval) prevents flapping, since every new replica pays a model load. `max_replicas=3` comes from the CPU budget, since the canary carries only 10 to 20% of traffic and three 0.25 CPU replicas keep the whole application within 2 CPUs, and `min_replicas=1` keeps the canary warm.

Part D: Release gate

D1.1 `partD/smoke_eta.py -url ... -expect-version ... [-max-p95-ms 80]` runs seven checks and exits 0 only if all pass: (1) `/healthz` returns 200, (2) a valid batch returns one finite ETA between 0 and 1000 min per row with the expected `model_version`, (3) a longer distance does not give a shorter ETA, (4) heavier traffic does not shorten the ETA, (5) more stops do not shorten the ETA, (6) invalid input (hour 25, and the infeasible 200 km with 30 stops) returns 422, and (7) p95 of 50 single-row requests is within the limit. Against the Part A v1 server, `-expect-version v1` passed 7 of 7 with exit code 0 (distance 2 to 100 km gave 24.3 to 277.1 min, traffic 0 to 1 gave 83.7 to 126.5 min, stops 1 to 20 gave 74.9 to 157.1 min, p95 3.4 ms). `-expect-version v2` against the same server failed the version check and exited with code 1, which shows the gate blocks a release when the wrong model is serving.

D1.2 Abort criteria for a 10% canary of v2 (each compares the v2 arm with the v1 arm over the same window).

Metric	Abort if	Window
HTTP 5xx error rate of v2	above 1% absolute, or above 2 times the prod rate	rolling 5 min, at least 200 requests
p95 latency of v2	above 80 ms, or above 1.5 times prod p95	3 consecutive 1 min windows
Online MAE of v2 against actual delivery times, overall and on rush-hour and rain slices	overall MAE worse than prod, or any slice more than 5% worse than prod	rolling 1 h, at least 300 completed deliveries
Share of v2 ETAs outside 5 to 600 min (implausible predictions)	above 0.5% of v2 responses	rolling 15 min

Not tested

- The smoke test was run only against the Part A v1 server, not against v2 or the Ray application.
- Ray autoscaling under load was configured and the deployment was healthy, but no load test was run against the Ray application, so scaling up to 3 replicas was not observed.
- The two zero-downtime update methods in B3 (blue-green or rolling replacement and hot reload) are described, not implemented.
- The abort criteria in D1.2 are designed, not exercised in a real canary.
- `/metrics` with two workers is per process. Aggregation across workers was not implemented or tested.
- Docker or Kubernetes deployment (for example `mlflow models build-docker`) was not run.